



CC1002 - Introducción a la Programación

CLASE 7 : TESTING Y ASEGURAMIENTO DE CALIDAD

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Nadia Decar (nadiadecar@ug.uchile.cl)

Eduardo Godoy (egodoy@dcc.uchile.cl)



¡1 es igual a 2!

Demostración



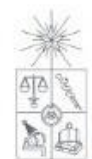
dcc

CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CHILE

¿1 == 2?

```
a = 1
```

```
b = 1
```



¿1 == 2?

```
a = 1
```

```
b = 1
```

```
a == b
```

```
a2 == ab
```

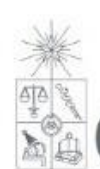
```
# se multiplica por a
```

```
# se le resta b2
```



¿1 == 2?

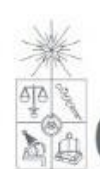
```
a = 1
b = 1
a == b                                # se multiplica por a
a2 == ab                             # se le resta b2
a2 - b2 == ab - b2                 # se factoriza
(a + b) (a - b) == b (a - b)          # se simplifica por (a - b)
(a + b) (a - b) == b (a - b)
(a + b) == b
```



¿1 == 2?

```
a = 1
b = 1
a == b                                # se multiplica por a
a2 == ab                             # se le resta b2
a2 - b2 == ab - b2                 # se factoriza
(a + b) (a - b) == b (a - b)         # se simplifica por (a - b)
(a + b) (a - b) == b (a - b)
(a + b) == b                          # pero a = 1 y b = 1
1 + 1 == 1
2 == 1
```

¡Un test debe ser cuidadoso!



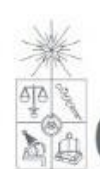
Problema

- El computador **no** tiene precisión infinita
 - No siempre hay un valor exacto
 - ¿Cómo asegurar un test?
- Hay situaciones en las que es necesario acotar el resultado

```
>>> import random
>>> # número al azar entre [0 - 1[
>>> random.random( )
⇒ 0.2965816702380957
```

EJERCICIO

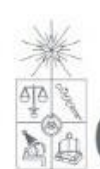
Construir una función que retorne números enteros al azar entre 1 y 10



Acotando un resultado

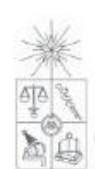
```
# escogeAlAzar: -> int
# Devuelve un número al azar entre 1 y 10
def escogeAlAzar( ):
    return int (10 * random.random() ) + 1
# Tests
```

¿Cómo sería el test dado que el número cambia?



Acotando un resultado

```
# escogeAlAzar: -> int
# Devuelve un número al azar entre 1 y 10
def escogeAlAzar( ):
    return int (10 * random.random() ) + 1
# Tests
assert escogeAlAzar( ) <= 10
assert escogeAlAzar( ) >= 1
```



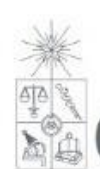
Cuando la precisión falla

```
>>> print ("1.0/2.0 =", 1.0/2.0)
⇒ 1.0/2.0 = 0.5

# copiamos el valor resultante y comparamos
>>> print (0.5 == 1.0/2.0)
⇒ True

>>>
⇒

>>>
⇒
```



Cuando la precisión falla

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```

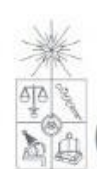
```
>>> print ("1.0/3.0 =", 1.0/3.0)
```

```
⇒ 1.0/3.0 = 0.333333333333
```

```
# copiamos el valor resultante y comparamos
```

```
>>> print (0.333333333333 == 1.0/3.0)
```

```
⇒ False
```



Cuando el resultado está “cerca”

```
>>> print (0.3 == 0.1+0.2)
```

```
⇒ False
```

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```



Cuando el resultado está “cerca”

```
>>> print (0.3 == 0.1+0.2)
      => False
>>> print (0.1 + 0.2)
      => 0.30000000000000004
```



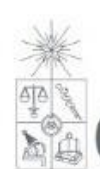
Humm... Hay que buscar un parche!

```
>>>
```

```
=>
```

```
>>>
```

```
=>
```



Cuando el resultado está “cerca”

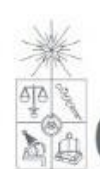
```
>>> print (0.3 == 0.1+0.2)
      => False

>>> print (0.1 + 0.2)
      => 0.30000000000000004

>>> print (abs(0.3 - (0.1+0.2)) < 0.001)
      => True

>>>
      =>
```

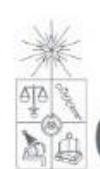
Humm... Hay que buscar un parche!



La función “cerca”

```
# cerca: num num num -> bool
# retorna True si x es igual a y con
# precisión epsilon

def cerca(x, y, epsilon) :
    diff = x - y
    return abs(diff) < epsilon
```



Cuando el resultado está “cerca”

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```

```
>>> print (cerca(0.3, (0.1+0.2), 0.001)
```

```
⇒ True
```



Cuando el resultado está “cerca”

```
>>>
```

```
⇒
```

```
>>>
```

```
⇒
```

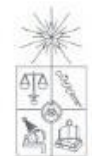
```
>>>
```

```
⇒
```

cerca(*x*, *y*, *epsilon*)

```
>>> print (cerca(0.3, (0.1+0.2), 0.001)
```

```
⇒ True
```

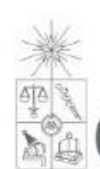


Series: con error

- Corrija el test de la función masmenos1divn:

- De manera que pueda validar, por ejemplo:
masmenos1divn(5)

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots \frac{1}{n}$$



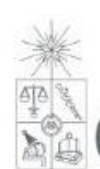
masmenos1divn

```
# masmenos1divn: int -> num
# dado un numero n, determina el valor de
# la serie 1-1/2+1/3-1/4+ . . . 1/n
# Ejemplo: masmenos1divn (1)  retorna 1
# Ejemplo: masmenos1divn (5)  retorna 0.783333

def masmenos1divn(n) :
    if n == 1 :
        return 1
    else :
        if espar(n) :
            return (-1.0/n) + masmenos1divn(n - 1)
        else:
            return (1.0/n) + masmenos1divn(n - 1)

# Test
assert masmenos1divn(1) == 1
assert masmenos1divn(2) == 0.5
assert . . .
```

← Rehacer el test

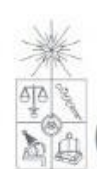


masmenos1divn

```
# masmenos1divn: int -> num
# dado un numero n, determina el valor de
# la serie 1-1/2+1/3-1/4+ . . . 1/n
# Ejemplo: masmenos1divn (1)  retorna 1
# Ejemplo: masmenos1divn (5)  retorna 0.783333
def masmenos1divn(n) :
    if n == 1 :
        return 1
    else :
        if espar(n) :
            return (-1.0/n) + masmenos1divn(n - 1)
        else:
            return (1.0/n) + masmenos1divn(n - 1)

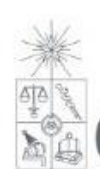
# Test
assert cerca(masmenos1divn(1), 1, 0.001)
assert cerca(masmenos1divn(2), 0.5, 0.001)
assert cerca(masmenos1divn(5), 0.783333, 0.001)
```

```
# cerca: num num num -> bool
# retorna True si x es igual a y con
# precisión epsilon
def cerca(x, y, epsilon) :
    diff = x - y
    return abs(diff) < epsilon
```



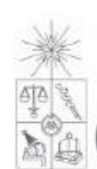
Raíz Cuadrada

- ¿Cómo calcular una raíz cuadrada?
 - Sin la calculadora 😞
- Se puede hacer “a mano”
 - Pero toma tiempo 😞
 - Usemos el computador 🤖
- Método creado por Herón
 - Matemático griego
 - Siglo I



Raíz Cuadrada: Herón

- ¿Cómo calcular la raíz cuadrada de **N**?
- Sea **N** = $a * b$
- 1. Se calcula el promedio p :
$$p = (a + b) / 2$$
- 2. Se calcula la razón r :
$$r = (a*b) / p$$
- 3. Si $\text{abs}(p - r) < \text{épsilon}$ la raíz es r o p
- 4. En caso contrario
 $a = p$
 $b = r$
Se repite al paso 1



Raíz Cuadrada: Herón

$$\sqrt{a * b}$$

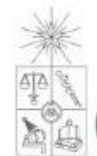
$a = 12$



$b = 3$

Un lado se reemplaza por el promedio (p)
El otro lado se reemplaza por $(a*b)/p$

$$a * b = 12 * 3 = 36$$



Raíz Cuadrada: Herón

$$\sqrt{a * b}$$

$a = 12$



$b = 3$

Un lado se reemplaza por el promedio (p)
El otro lado se reemplaza por $(a*b)/p$

$$\begin{aligned} a * b &= 12 * 3 = 36 \\ a &= (12 + 3) / 2 = 15 / 2 = 7.5 \\ b &= 36 / 7.5 = 4.8 \\ a * b &= 7.5 * 4.8 = 36 \end{aligned}$$



Raíz Cuadrada: Herón

$$\sqrt{a * b}$$

$$a = 12$$



$$b = 3$$

Un lado se reemplaza por el promedio (p)
El otro lado se reemplaza por (a*b)/p

$$\begin{aligned} a * b &= 12 * 3 = 36 \\ a &= (12 + 3) / 2 = 15 / 2 = 7.5 \\ b &= 36 / 7.5 = 4.8 \\ a * b &= 7.5 * 4.8 = 36 \end{aligned}$$

$$a = 7.5$$



$$b = 4.8$$



Raíz Cuadrada: Herón

$a = 7.5$



$b = 4.8$

$$\sqrt{a * b}$$

Un lado se reemplaza por el promedio (p)
El otro lado se reemplaza por $(a*b)/p$

$$\begin{aligned} a &= (7.5+4.8) / 2 = 12.3 / 2 = 6.15 \\ b &= 36 / 6.15 = 5.853658... \end{aligned}$$



Raíz Cuadrada: Herón

$$\sqrt{a * b}$$

$a = 7.5$



$b = 4.8$

Un lado se reemplaza por el promedio (p)
El otro lado se reemplaza por $(a*b)/p$

```
a = (7.5+4.8) / 2 = 12.3 / 2 = 6.15  
b = 36 / 6.15 = 5.853658...
```

$a = 6.15$



$b = 5.853658...$

Esto se repite hasta que a y b
estén bastante "cerca"

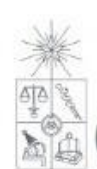


raizCuadrada(x, epsilon)

```
# raizCuadrada: num num -> num
# retorna la raíz cuadrada con
# una precisión epsilon

def raizCuadrada(x, epsilon) :
    a = x
    b = 1
    # notar que (a * b) == (x * 1) == x
    return heron(a, b, epsilon)

# Test
```

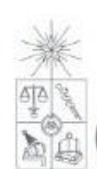


heron(a, b, epsilon)

```
# heron: num num float -> float
# retorna la raíz cuadrada con
# una precisión epsilon usando el método heron

def heron(a, b, epsilon) :
    p = (a + b) / 2.0
    r = (a * b) / p      # x == (a*b)
```

- ¿Cómo calcular la raíz cuadrada de **N**?
- Sea **N** = $a * b$
- 1. Se calcula el promedio p :
 $p = (a + b) / 2$
- 2. Se calcula la razón r :
 $r = (a * b) / p$
- 3. Si $\text{abs}(p - r) < \text{épsilon}$ la raíz es r o p
- 4. En caso contrario
 $a = p$
 $b = r$
Se repite al paso 1



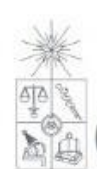
heron(a, b, epsilon)

```
# heron: num num float -> float
# retorna la raíz cuadrada con
# una precisión epsilon usando el método heron

def heron(a, b, epsilon) :
    p = (a + b) / 2.0
    r = (a * b) / p          # x == (a*b)

    if cerca(p, r, epsilon) :
        return r             # tambien puede ser p
```

- ¿Cómo calcular la raíz cuadrada de **N**?
- Sea **N** = $a * b$
- 1. Se calcula el promedio p :
 $p = (a + b) / 2$
- 2. Se calcula la razón r :
 $r = (a * b) / p$
- 3. Si $\text{abs}(p - r) < \text{épsilon}$ la raíz es r o p
- 4. En caso contrario
 $a = p$
 $b = r$
Se repite al paso 1



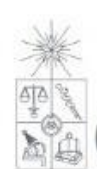
heron(a, b, epsilon)

```
# heron: num num float -> float
# retorna la raíz cuadrada con
# una precisión epsilon usando el método heron

def heron(a, b, epsilon) :
    p = (a + b) / 2.0
    r = (a * b) / p      # x == (a*b)

    if cerca(p, r, epsilon) :
        return r        # tambien puede ser p
    else :
        return heron(p, r, epsilon)
```

- ¿Cómo calcular la raíz cuadrada de **N**?
- Sea **N** = $a * b$
- 1. Se calcula el promedio **p**:
 $p = (a + b) / 2$
- 2. Se calcula la razón **r**:
 $r = (a * b) / p$
- 3. Si $\text{abs}(p - r) < \text{épsilon}$ la raíz es **r** o **p**
- 4. En caso contrario
 a = **p**
 b = **r**
 Se repite al paso 1



raizCuadrada(x, epsilon)

```
# raizCuadrada: num num -> num
# retorna la raíz cuadrada con
# una precisión epsilon

def raizCuadrada(x, epsilon) :
    a = x
    b = 1

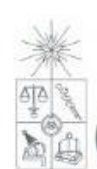
    # notar que (a * b) == (x * 1) == x
    return heron(a, b, epsilon)

# Test
import math
assert cerca(raizCuadrada(2, 0.1), math.sqrt(2), 0.1)
assert cerca(raizCuadrada(4, 0.01), math.sqrt(4), 0.01)
```

```
# heron: num num float -> float
# retorna la raíz cuadrada con
# una precisión epsilon usando el método heron

def heron(a, b, epsilon) :
    p = (a + b) / 2.0
    r = (a * b) / p      # x == (a*b)

    if cerca(p, r, epsilon) :
        return r        # tambien puede ser p
    else :
        return heron(p, r, epsilon)
```

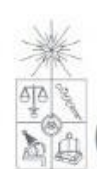


```
>>> import math
>>> print raizCuadrada(2, 0.01), "sqrt=", math.sqrt(2)
⇒ 1.41666666667 sqrt= 1.41421356237

>>> print raizCuadrada(2, 0.0001), "sqrt=", math.sqrt(2)
⇒ 1.41421568627 sqrt= 1.41421356237

>>> print raizCuadrada(25, 0.01), "sqrt=", math.sqrt(25)
⇒ 5.00001295305 sqrt= 5.00

>>> print raizCuadrada(0.1, 0.01), "sqrt=", math.sqrt(0.1)
⇒ 0.32 54 sqrt= 0.316227766017
```



Buscando un valor terminal

- El número π es infinito

- Serie de Leibniz para el cálculo de π

3.1415926535897932384626

- Hasta cuándo calcular para obtener π ?
 1. Hasta el término k
 2. Hasta un error ε

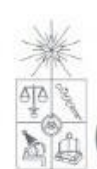
$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

π calculado para k términos

- Paso 1: crear una función que calcule el valor de la serie en k:

$$\text{leibniz_kesimo} = (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

- Paso 2: calcular la serie para los k términos

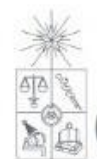


leibniz_kesimo(k)

```
# leibniz_kesimo: num -> num
# dado un valor de k, calcula el término k
# de la serie de leibniz.

def leibniz_kesimo(k) :
    if esPar(k) :
        signo = -1
    else :
        signo = 1
    # se calcula el valor k-ésimo
    valor = signo * (1 / (2.0 * k - 1))
    return valor
```

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

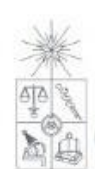


π calculado para k términos

- Paso 1: crear una función que calcule el valor de la serie en k:

$$\text{leibniz_kesimo} = (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

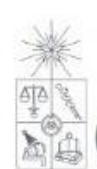
- Paso 2: calcular la serie para los k términos



pi_Leibniz(k)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

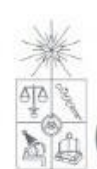
```
# pi_Leibniz(n): int -> num
# calcula el número pi hasta la iteración k
def pi_Leibniz(k) :
    if (k == 1) :
        return 1
```



pi_Leibniz(k)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

```
# pi_Leibniz(n): int -> num
# calcula el número pi hasta la iteración k
def pi_Leibniz(k) :
    if (k == 1) :
        return 1
    else :
        # se calcula el valor k
        valorK = leibniz_kesimo(k)
```

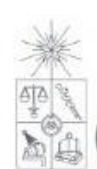


pi_Leibniz(k)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

```
# pi_Leibniz(n): int -> num
# calcula el número pi hasta la iteración k
def pi_Leibniz(k) :
    if (k == 1) :
        return 1
    else :
        # se calcula el valor k
        valorK = leibniz_kesimo(k)

        # se suma el valor a la serie y
        # se calcula los términos faltantes
        return pi_Leibniz(k - 1) + valorK
```



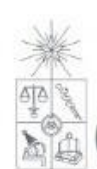
pi_Leibniz(k)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$


```
# pi_Leibniz(n): int -> num
# calcula el número pi hasta la iteración k
def pi_Leibniz(k) :
    if (k == 1) :
        return 1
    else :
        # se calcula el valor k
        valorK = leibniz_kesimo(k)

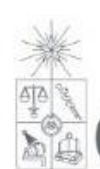
        # se suma el valor a la serie y
        # se calcula los términos faltantes
        return pi_Leibniz(k - 1) + valorK
```

```
# leibniz_kesimo: num -> num
# dado un valor de k, calcula el
# término k de la serie de leibniz.
def leibniz_kesimo(k) :
    if esPar(k) :
        signo = -1
    else :
        signo = 1
    # se calcula el valor k-ésimo
    valor = signo * (1 / (2.0 * k - 1))
    return valor
```



Pi(K)

```
# Pi: int -> num
# retorna el número Pi luego de
# haber calculado la serie de Leibniz hasta k
def Pi(k) :
    return 4 * pi_Leibniz(k)
```



Resultados de Pi(k)

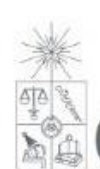
```
>>> print Pi(1)
⇒ 4

>>> print(Pi(2))
⇒ 2.66666666667

>>> print(Pi(100))
⇒ 3.13159290356

>>> print(Pi(999))
⇒ 3.14259365434
```

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Buscando un valor terminal

- El número π es infinito

[illegible]

- Serie de Leibniz para el cálculo de π

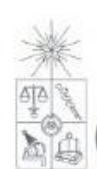
$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

- Hasta cuándo calcular para obtener π ?
 1. Hasta el término k
 2. Hasta un error ε

Buscando un valor terminal

- Propuesta de solución:
 - Calcular π_k
 - Calcular π_{k+1}
 - Si $(\pi_k - \pi_{k+1}) < \varepsilon$ se retorna π_{k+1}
 - En caso contrario se incorpora otro término


$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Buscando un valor terminal

- Propuesta de solución:
 - Calcular π_k
 - Calcular π_{k+1}
 - Si $(\pi_k - \pi_{k+1}) < \varepsilon$ se retorna π_{k+1}
 - En caso contrario se incorpora otro término

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} < \varepsilon$$


$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Buscando un valor terminal

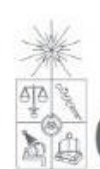
- Propuesta de solución:
 - Calcular π_k
 - Calcular π_{k+1}
 - Si $(\pi_k - \pi_{k+1}) < \varepsilon$ se retorna π_{k+1}
 - En caso contrario se incorpora otro término

$$\frac{\pi}{4} = 1 - \frac{1}{3} < \varepsilon$$

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} < \varepsilon$$

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} < \varepsilon$$

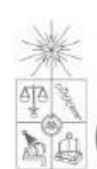

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Pi(epsilon)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

```
# Pi: num -> num
# retorna el número Pi con
# una precisión epsilon
def Pi(epsilon) :
    k = 1      # valor de k al partir
    pik = 1    # valor de la serie cuando k vale 1
    return 4 * leibniz(k, pik, epsilon)
```



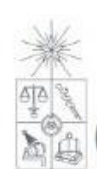
leibniz(k, pik, epsilon)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

```
# cerca: num num -> num
# dado un valor de Pi en la iteración k de la serie
# de leibniz, calcula el término k+1
# de la serie de leibniz. Si (pik - pik+1) < épsilon
# retorna pik+1
```

```
def leibniz(k, pik, epsilon) :
    # se calcula el valor de k+1
    pikmas1 = pik + leibniz_ksimo(k+1)
```

```
# leibniz_ksimo: num -> num
# dado un valor de k, calcula el
# término k de la serie de leibniz.
def leibniz_ksimo(k) :
    if esPar(k) :
        signo = -1
    else :
        signo = 1
    # se calcula el valor k-ésimo
    valor = signo * (1 / (2.0 * k - 1))
    return valor
```



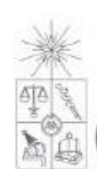
leibniz(k, pik, epsilon)

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$

```
# cerca: num num -> num
# dado un valor de Pi en la iteración k de la serie
# de leibniz, calcula el término k+1
# de la serie de leibniz. Si (pik - pik+1) < epsilon
# retorna pik+1
```

```
def leibniz(k, pik, epsilon) :
    # se calcula el valor de k+1
    pikmas1 = pik + leibniz_ksimo(k+1)

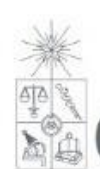
    if cerca(pik, pikmas1, epsilon) :
        return pikmas1
    else :
        return leibniz(k+1, pikmas1, epsilon)
```



Resultados de Pi(epsilon)

```
>>> print(Pi(0.5))  
⇒ 2.66666666667  
  
>>> print(Pi(0.1))  
⇒ 2.97604617605  
  
>>> print(Pi(0.01))  
⇒ 3.16119861299  
  
>>> print(Pi(0.001))  
⇒ 3.14358865959
```

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Resultados de Pi(epsilon)

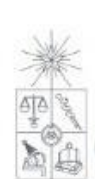
```
>>> print(Pi(0.5))  
⇒ 2.66666666667  
>>> print(Pi(0.1))  
⇒ 2.97604617605  
>>> print(Pi(0.01))  
⇒ 3.16119861299  
>>> print(Pi(0.001))  
⇒ 3.14358865959
```

Notar que épsilon es 0.1,
pero:

$$|\text{Pi}(0.1) - \pi| > 0.1$$

¿A qué se debe?

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Resultados de Pi(epsilon)

```
>>> print(Pi(0.5))
      => 2.66666666667
>>> print(Pi(0.1))
      => 2.97604617605
>>> print(Pi(0.01))
      => 3.16119861299
>>> print(Pi(0.001))
      => 3.14358865959
```

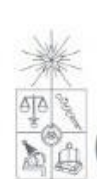
Notar que épsilon es 0.1,
pero:

$$|\text{Pi}(0.1) - \pi| > 0.1$$

¿A qué se debe?

épsilon es la distancia
entre leibniz_k y leibniz_{k+1}
pero no la distancia a π

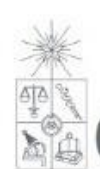
$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \square + (-1)^{k+1} * \frac{1}{(2 * k) - 1}$$



Ejercicio 1

Encuentre el error, dé un ejemplo donde quede claro el error encontrado y escriba el código correcto.

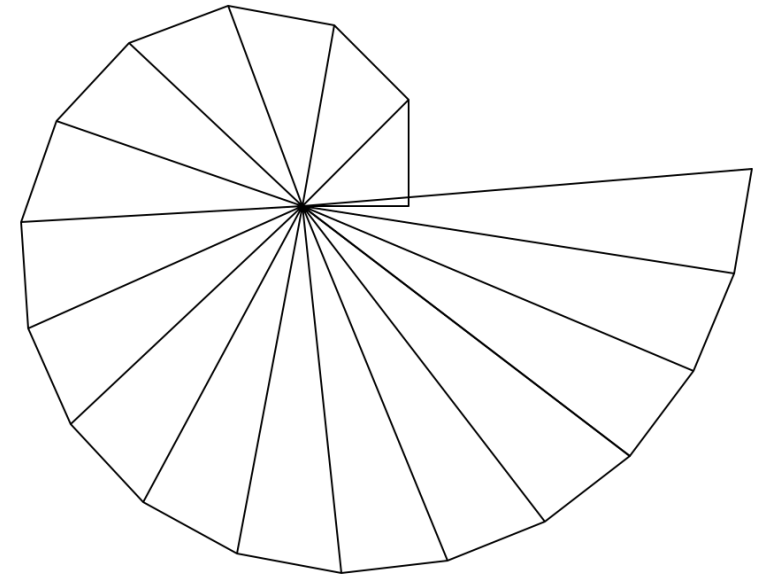
```
def esPar(n):  
    if n % 2 == 0 and n > 0:  
        resultado = "Par positivo"  
    elif n % 2 != 0 and n > 0:  
        resultado = "Impar positivo"  
    elif n % 2 == 0 or n < 0:  
        resultado = "Par negativo"  
    elif n == 0:  
        resultado = "Cero"  
    else:  
        resultado = "Impar negativo"  
    return resultado
```



Ejercicio 2: Hemos sido hackeados

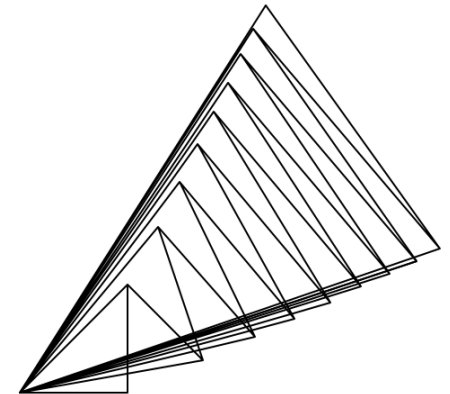
Espiral de Teodoro con turtle

- Cada triángulo rectángulo levanta un cateto de largo 1 sobre la hipotenusa anterior.
- La hipotenusa n mide la raíz cuadrada de $n + 1$.
- El dibujo se genera por recursión: un triángulo por llamada.



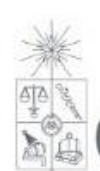
Un hacker modificó el código, y ahora el programa genera la siguiente imagen

Encuentre el error y corrijalo.



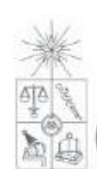
Ejercicio 2: preparación

```
import math
import turtle
# Configurar pantalla y tortuga
screen = turtle.Screen()
screen.setup(800, 800)
screen.title("Espiral de Teodoro (Version Recursiva)")
t = turtle.Turtle()
t.speed(2)    # velocidad moderada, para que se vea el proceso de dibujo
t.pensize(1)
# Escala en pixeles para la unidad de medida (1 unidad = 60 px)
escala = 60
# Centro del abanico
origen_x, origen_y = 0, 0
# Numero de triangulos a dibujar en el abanico
numero_triangulos = 30
```



Ejercicio 2: la función recursiva

```
# espiral_teodoro: turtle num num int int -> None
# dibuja recursivamente el triangulo numero i del abanico de Teodoro
# y se llama a si misma para dibujar el siguiente triangulo, usando
# como nuevo cateto base el extremo exterior del triangulo actual
#   x_actual, y_actual -> vertice exterior del triangulo anterior
#   i                   -> indice del triangulo actual (1, 2, 3, ...)
#   numero_triangulos  -> cuantos triangulos se quieren dibujar
def espiral_teodoro(t, x_actual, y_actual, i, numero_triangulos):
    # Caso base: ya dibujamos todos los triangulos pedidos
    if i > numero_triangulos:
        return
    # Angulo del cateto base actual respecto al origen
    angulo_base = math.atan2(y_actual, x_actual)
    # El nuevo cateto unitario se levanta a 90 grados de la hipotenusa
    angulo_cateto_nuevo = angulo_base + math.pi / 2
    # Coordenadas del nuevo vertice exterior (cateto de longitud 1)
    y_nuevo = x_actual + escala * math.cos(angulo_cateto_nuevo)
    x_nuevo = y_actual + escala * math.sin(angulo_cateto_nuevo)
    # Dibuja el triangulo actual
    t.goto(x_nuevo, y_nuevo)      # dibuja el cateto de longitud 1
    t.goto(origen_x, origen_y)   # cierra el triangulo volviendo al centro
    t.goto(x_nuevo, y_nuevo)     # se posiciona en el extremo exterior
    # Llamada recursiva para dibujar el siguiente triangulo
    espiral_teodoro(t, x_nuevo, y_nuevo, i + 1, numero_triangulos)
```



Ejercicio 2: dibujo y ejecución

```
# Dibujar el primer triangulo rectangulo (Base = 1, Altura = 1)
t.penup()
t.goto(origen_x, origen_y)
t.pendown()
t.goto(origen_x + escala, origen_y)
# Iniciamos la recursion desde el triangulo numero 1
espiral_teodoro(t, origen_x + escala, origen_y, 1, numero_triangulos)
t.hideturtle()
turtle.done()
```

